

Algoritma DFS

Dalam tutorial ini, Anda akan belajar apa itu algoritma DFS

Traversal berarti mengunjungi semua simpul graph. Depth first traversal atau Depth first Search adalah algoritma rekursif untuk mencari semua simpul dari graph atau struktur data pohon

Algoritma DFS

Implementasi DFS standar menempatkan setiap simpul grafik ke dalam salah satu dari dua kategori:

1. Dikunjungi
2. Tidak Dikunjungi

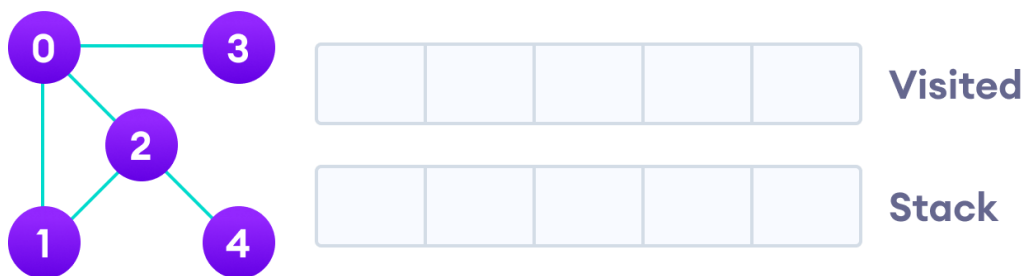
Tujuan dari algoritma ini adalah untuk menandai setiap simpul yang dikunjungi sambil menghindari siklus.

Algoritma DFS bekerja sebagai berikut:

1. Mulailah dengan meletakkan salah satu dari simpul graph di atas tumpukan (stack).
2. Ambil item teratas dari tumpukan dan tambahkan ke daftar yang dikunjungi (visited).
3. Buat daftar node yang berdekatan simpul itu. Tambahkan yang tidak ada dalam daftar yang dikunjungi ke bagian atas tumpukan.
4. Terus ulangi langkah 2 dan 3 sampai tumpukan kosong.

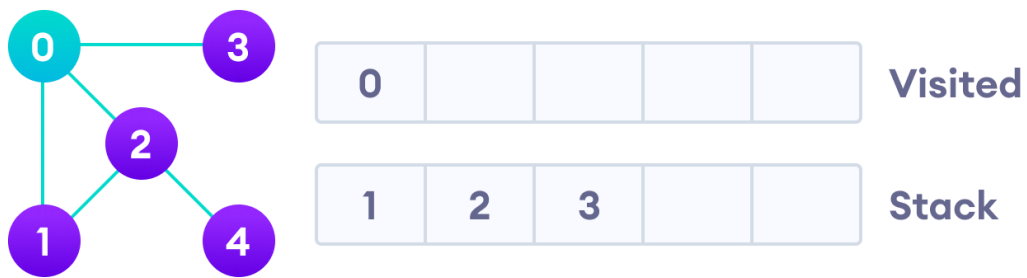
Contoh DFS

Mari kita lihat bagaimana algoritma Pencarian Kedalaman Pertama bekerja dengan sebuah contoh. Kita menggunakan graph tidak terarah dengan 5 simpul.



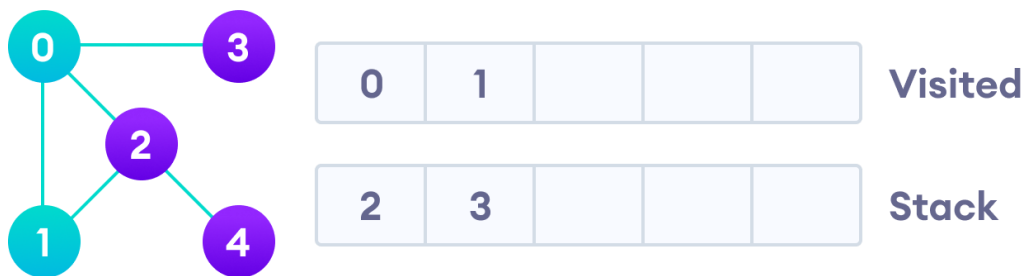
Graph tidak berarah dengan 5 simpul

Kita mulai dari titik 0, algoritma DFS dimulai dengan meletakkannya di daftar yang Dikunjungi dan menempatkan semua simpul yang berdekatan di tumpukan.



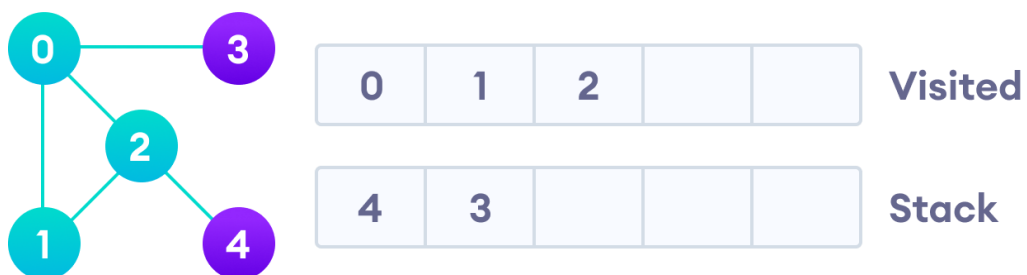
Kunjungi elemen dan letakkan di daftar yang dikunjungi

Selanjutnya, kita mengunjungi elemen di atas tumpukan yaitu 1 dan pergi ke node yang berdekatan. Karena 0 sudah dikunjungi, kita malah mengunjungi 2.

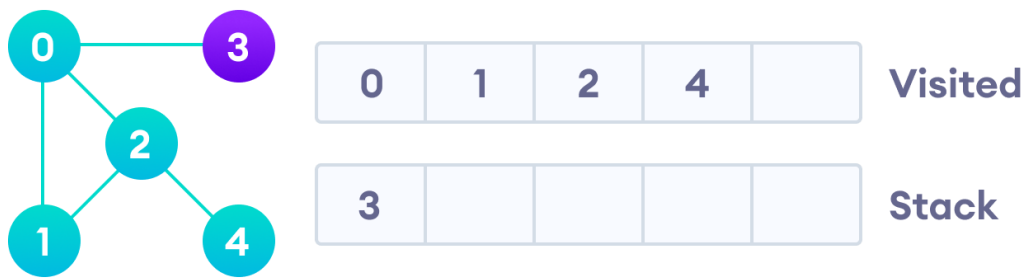


Kunjungi elemen di bagian atas tumpukan

Vertex 2 memiliki simpul berdekatan yang belum dikunjungi di 4, jadi kita menambahkannya ke atas tumpukan dan mengunjunginya.



Vertex 2 memiliki simpul berdekatan yang belum dikunjungi di 4, jadi kita menambahkannya ke atas tumpukan dan mengunjunginya.



Setelah kita mengunjungi elemen terakhir 3, ia tidak memiliki node yang berdekatan yang belum dikunjungi, jadi kita telah menyelesaikan Traversal Pertama Kedalaman dari graph.

Pseudocode DFS (implementasi rekursif)

Pseudocode untuk DFS ditunjukkan di bawah ini. Dalam fungsi `init()`, perhatikan bahwa kita menjalankan fungsi DFS pada setiap node. Ini karena grafik mungkin memiliki dua bagian terputus yang berbeda sehingga untuk memastikan bahwa kita menutupi setiap simpul, kita juga dapat menjalankan algoritma DFS pada setiap simpul.

```
DFS(G, u)
    u.visited = true
    for each v ∈ G.Adj[u]
        if v.visited == false
            DFS(G,v)

init() {
    For each u ∈ G
        u.visited = false
    For each u ∈ G
        DFS(G, u)
}
```

Contoh Python

Kode untuk Algoritma Pencarian Kedalaman Pertama dengan contoh ditunjukkan di bawah ini. Kode telah disederhanakan sehingga kita dapat fokus pada algoritme daripada detail lainnya.

```
# DFS algorithm in Python

# DFS algorithm
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
```

```
visited.add(start)

print(start)

for next in graph[start] - visited:
    dfs(graph, next, visited)
return visited

graph = {'0': set(['1', '2']),
        '1': set(['0', '3', '4']),
        '2': set(['0']),
        '3': set(['1']),
        '4': set(['2', '3'])}

dfs(graph, '0')
```

Aplikasi Algoritma DFS

1. Untuk menemukan jalan
2. Untuk menguji apakah graphnya bipartit
3. Untuk menemukan komponen graph yang sangat terhubung
4. Untuk mendeteksi siklus dalam graph

Referensi :

<https://www.programiz.com/dsa/graph-dfs>